EXPERIMENT 14

Exploring Pro-Code Extension of a Zoho Creator Application using Zoho Catalyst / Custom Functions

Name: Jenell S G | Register Number: 240701212

Aim

To explore how pro-code capabilities can extend a low-code Zoho Creator application using Zoho Catalyst and custom functions, by creating and executing a custom Deluge function that calculates the e-waste priority score in the EcoCycle application.

Introduction

Low-code platforms let applications be developed with visual components such as forms, reports and workflows. This makes it quick to build a standard application such as EcoCycle, where items are recorded, prioritised, collected and recycled.

When the standard functionality is insufficient, pro-code extensions allow developers to add programmable logic. Zoho Creator supports this through Deluge and through **custom functions**, which are reusable units of Deluge code that can be called wherever they are needed. Zoho Catalyst, Zoho's serverless development platform, is a further pro-code option for requirements that go beyond what can be scripted inside the application.

In EcoCycle the priority rule is written directly inside the E-Waste Entry workflow. In this experiment the same rule is placed in a custom function named **calculatePriorityScore** so that it can be reused and tested on its own. The function uses the Deluge language and the Default namespace, takes four arguments and returns a number.

Pro-Code Extension in Zoho Creator

Extending a low-code application with custom functions means that the visual structure (forms, reports and workflows) is retained, while specific logic is written as code in a function and used where required.

Concept	Description	Use in EcoCycle
Custom function	A named, reusable function created in the Functions section of the Workflow area	calculatePriorityScore
Language	Deluge, or another language where the platform offers it	Deluge
Namespace	A group that holds related functions; Default is used here	Default
Arguments	Values passed to the function when it is called	quantity, age, hazardous, condition
Return type	The type of value returned (void returns nothing)	Number (the score)
Zoho Catalyst	Serverless platform for logic that cannot be scripted inside Creator	Possible future use, with the planned REST API integration

Software / Tools Required

- Zoho Creator with the EcoCycle application open in Edit mode
- Web browser with an active internet connection

Procedure

1. Open the EcoCycle application in Zoho Creator in Edit mode.
2. Navigate to the **Workflow** section (Figure 1).
3. Open the **Functions** section from the tabs of the Workflow area (next to Batch workflows) and select the option to create a new function.
4. The **Execute a function when called** form is displayed with the fields Name the function, Select the language, Select namespace, Select return type and Specify arguments (Figure 2).
5. Enter the function name **calculatePriorityScore**.
6. Select **Deluge** as the programming language and use the **Default** namespace.
7. Set the return type to **Number**.
8. Add the four arguments: quantity (Number), age (Number), hazardous (Text) and condition (Text). Select **Create Function**.
9. The Deluge editor opens with the function declaration generated from the selected options (Figure 3). The editor lists ready-made statements such as Condition, Data Access and Miscellaneous on the left.
10. Write the body of the function as in Listing 14.1 and click **Save**.
11. Select **Execute**, enter the test values of Table 14.1 in the Execute window and read the returned value (Figure 4).
12. Confirm that the function is listed in the Functions section.

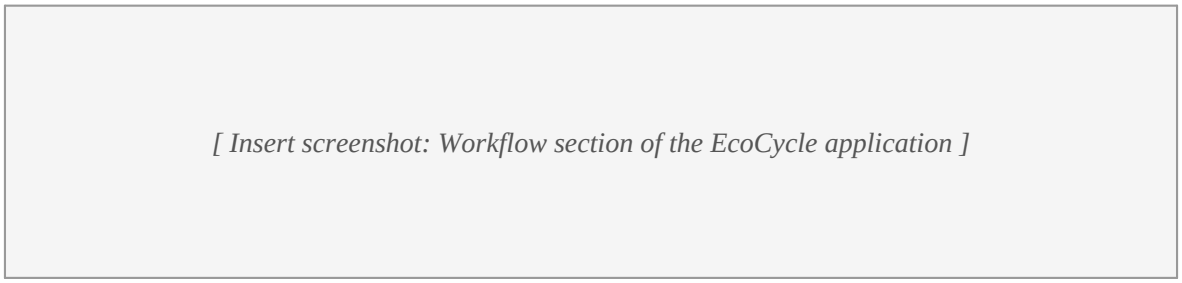

Figure 1: Workflow Section of the Application

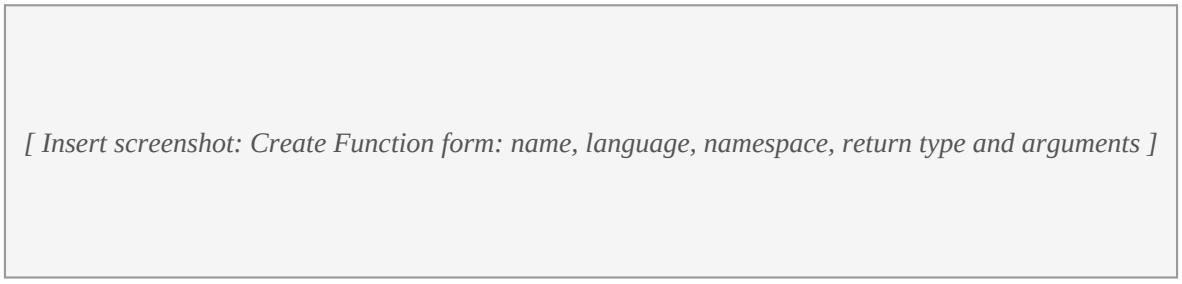

Figure 2: Create Function Form – Name, Language, Namespace, Return Type and Arguments

Figure 3: Deluge Editor with the Function Declaration and Body

Listing 14.1: Custom function calculatePriorityScore (Default namespace, return type Number)

```
int calculatePriorityScore(int quantity, int age, string hazardous, string condition)
{
    score = 0;
    // Quantity
    if(quantity >= 8)       { score = score + 3; }
    else if(quantity >= 4)  { score = score + 2; }
    else if(quantity >= 2)  { score = score + 1; }
    // Age in years
    if(age >= 3)            { score = score + 2; }
    else if(age >= 1)       { score = score + 1; }
    // Hazardous material
    if(hazardous == "Yes")  { score = score + 3; }
    // Condition
    if(condition == "Not Working" || condition == "Damaged")
    {
        score = score + 2;
    }
    return score;
}
```

Note: the weights are the reference scheme of the project report (Appendix C), written as a function so that it can be reused; adjust them to the weights used in the application.

Calling the function from the workflow

The E-Waste Entry workflow of Experiment 6 can call the function in place of the repeated code, and the level can then be set from the returned score:

Listing 14.2: Calling the custom function

```
score = thisapp.calculatePriorityScore(input.Quantity, input.Age_years,
                                       input.Contains_Hazardous_Material, input.Condition);
if(score >= 8)      { level = "High"; }
else if(score >= 4) { level = "Medium"; }
else                { level = "Low"; }
```

Test Values

The function was executed with the four stored e-waste entries. The expected return values of the reference scheme are shown; the value reported by the Execute window should be recorded next to each.

Test	Quantity	Age	Hazardous	Condition	Expected score	Level
laptop	5	3	No	Working	4	Medium
computer	2	1	No	Working	2	Low
Printer	8	5	Yes	Not Working	10	High
monitor	10	5	Yes	Damaged	10	High

Table 14.1: Test values for calculatePriorityScore

[Insert screenshot: Execute window showing the returned score and the message Executed Successfully]

Figure 4: Execute Window – Function Executed Successfully

Observations

- The function was created with the Deluge language, the Default namespace, a Number return type and four arguments.
- The function declaration is generated automatically by the editor from the selected options.
- The same function can be called from any workflow, which removes repeated code and gives one place to change the weights.
- The function implements one rule only. It does not process several records, call other systems or run on a schedule.
- The Creator REST API integration and any Zoho Catalyst service are planned for the future and are not part of the delivered EcoCycle application.

Result

The custom Deluge function calculatePriorityScore was created in Zoho Creator using the Default namespace and a Number return type, saved, listed in the Functions section and executed with the test entries of EcoCycle. It returns the same scores as the reference scheme used by the E-Waste Entry workflow.

Conclusion

Low-code platforms such as Zoho Creator simplify application development by providing forms, reports and workflows that can be assembled visually. Where specific requirements go beyond the standard components, pro-code extensions provide additional flexibility, and custom Deluge functions can introduce application-specific logic in a reusable form.

In this experiment a custom function was created, saved, executed and called from a workflow. Although the function was small, it shows the procedure for creating, executing and verifying a pro-code extension. Combining low-code development with such extensions, and with Zoho Catalyst where needed, supports a flexible application in which standard functionality is built visually and special logic, such as e-waste scoring rules, is added through code.